Md. Murad Ul Momin | Rima Saha | Sakia Tabassum Anonna | Ashraful Hossain Riyan
Group 06 | CSE 499A | Section 15 | Faculty: Mohammad Shifat-E-Rabbi

PRIVATE SHIELD: A Privacy-Preserving Federated Intelligence System for IoT Threat Detection

Abstract

Internet of Things (IoT) environments generate distributed and privacy-sensitive network data, while many edge devices have limited processing power and weak built-in security. Conventional cloud-based intrusion detection requires raw telemetry transfer, which introduces privacy leakage, communication cost, and dependence on a central service. This report proposes PRIVATE SHIELD, a unified federated intelligence framework for IoT threat detection. The framework keeps raw traffic at each client, trains lightweight local detection models, protects shared updates through differential privacy and gradient clipping, filters suspicious updates before aggregation, assigns reputation scores to clients, and presents human-readable alerts through a security operations center dashboard. The proposed design addresses four connected weaknesses found in existing research: gradient leakage, poisoning attacks, free-rider participation, and limited operational explainability. The report defines the architecture, workflow, main algorithms, implementation modules, threat model, and evaluation plan for an academic prototype. Expected results include lower raw-data exposure, improved tolerance to malicious clients, reduced communication compared with centralized telemetry collection, and faster interpretation of security events.

Index Terms—Federated learning, Internet of Things, intrusion detection, differential privacy, poisoning defense, free-rider detection, explainable security.

I. INTRODUCTION

IoT devices are now used in homes, healthcare, industrial monitoring, transportation, and smart infrastructure. Their wide deployment increases the attack surface because many devices run with limited memory, low-power processors, outdated firmware, or weak authentication. Botnets, denial-of-service traffic, scanning activity, and malicious device behavior can therefore spread quickly across connected environments.

A centralized intrusion detection system (IDS) normally transfers device telemetry to a cloud server for training and analysis. Although this approach simplifies management, it exposes sensitive traffic, consumes bandwidth, and creates a single point of failure. Federated learning (FL) offers an alternative by sending a model to clients, training locally, and sharing only model updates. However, FL does not automatically guarantee privacy or security. Gradients can leak information, malicious clients can poison the shared model, and free-riders can receive benefits without contributing reliable updates [7]–[12].

PRIVATE SHIELD is proposed as an edge-focused system that connects privacy, adversarial robustness, client reliability, and operational explanation in one workflow. The system is intended for an academic prototype rather than a production security appliance, but its modular structure supports controlled experiments and future extension.

II. PROBLEM STATEMENT AND RESEARCH MOTIVATION

The reviewed studies show that recent FL-based IDS methods improve distributed learning under heterogeneous IoT data [1]–[6]. Privacy-preserving methods add differential privacy, secure aggregation, or execution proofs [7], [8], [10], while robust aggregation methods identify suspicious model updates [6], [9], [10]. Separate studies investigate free-rider behavior [11], [12]. The central problem is that these mechanisms are commonly evaluated as isolated defenses. A practical IoT security workflow needs them to operate together without creating excessive computation or communication overhead.

A. Primary Problems

The proposed work focuses on six connected problems: (1) raw telemetry exposure in centralized IDS, (2) privacy leakage from shared gradients, (3) model poisoning by malicious clients, (4) unreliable or free-rider participation, (5) non-independent and imbalanced traffic across devices, and (6) weak translation of technical model outputs into understandable security alerts.

B. Design Objectives

The system is designed to keep raw records local, use lightweight models suitable for simulated edge clients, add controlled privacy noise, reject abnormal updates before aggregation, maintain a simple reputation score for each client, support non-IID experiments, and provide a live dashboard that explains why an event is considered suspicious. These objectives directly convert the identified research gaps into implementable software modules.

III. PROPOSED SYSTEM ARCHITECTURE

The architecture contains distributed IoT clients, a federated coordination server, an adversarial validation layer, and an SOC interface. Each client collects or receives local traffic records, preprocesses them, and trains a lightweight binary or multi-class classifier. Raw data never leaves the client. After local training, the client clips its model update, adds differential privacy noise, and sends the protected update with limited metadata to the server.

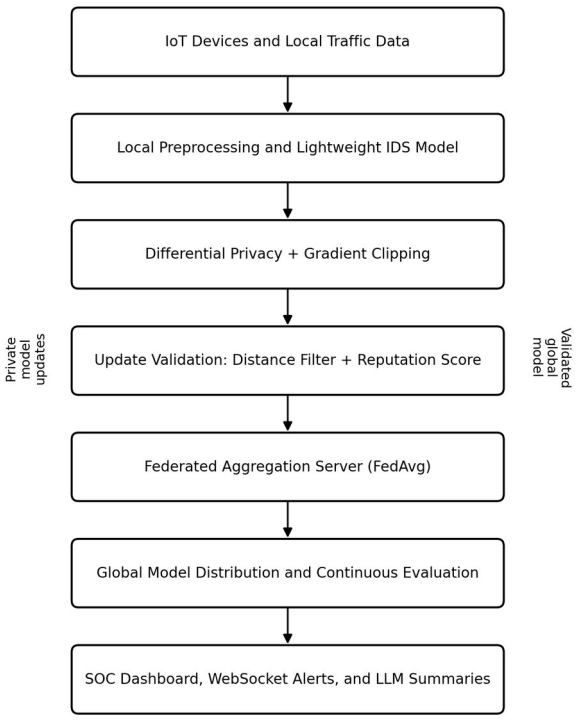

Fig. 1. Proposed PRIVATE SHIELD architecture and processing flow.

Before aggregation, the server compares each received update with a trusted reference such as the current global model, a robust center, or the median update. Euclidean distance provides a simple anomaly score for the initial prototype. Updates that exceed a configurable threshold are rejected or assigned a reduced weight. A reputation component records accepted, rejected, missing, and unstable participation so that consistently unreliable clients have less influence in future rounds.

Accepted updates are combined using Federated Averaging (FedAvg). The updated global model is returned to participating clients for the next round. Training statistics, alert counts, suspected attacks, client status, and model performance are transmitted through WebSockets to a React-based dashboard. An optional large language model module receives structured event summaries rather than raw private traffic and produces short forensic explanations for administrators.

IV. DETAILED PROPOSED METHODOLOGY

A. Local Data Preparation and Detection Model

Each simulated IoT client stores its local traffic in a separate partition. Features may include packet rate, byte count, protocol or port information, connection frequency, message interval, device type, and selected sensor values. Categorical fields are encoded and numerical features are scaled using parameters learned from the local training split. To represent real IoT diversity, client partitions will be intentionally non-IID: some clients will contain different device behaviors, normal ranges, or attack proportions.

The baseline classifier will be a small multilayer perceptron or another lightweight model that can be trained quickly on CPU. A centralized model will first establish an upper-bound reference. The same architecture will then be trained through FL so that differences in accuracy, recall, latency, and communication can be measured fairly.

B. Federated Training Workflow

At round t, the server selects available clients and sends the current global parameters. Client k trains for a small number of local epochs using its private dataset and returns an update. FedAvg calculates a weighted average based on the number of local training samples. Local epochs and batch size will remain small to avoid unrealistic edge computation. The server records round duration, selected clients, received updates, rejected updates, and global validation results.

C. Differential Privacy and Gradient Clipping

Differential privacy is introduced through Opacus or an equivalent privacy engine. Per-sample gradients are clipped to a maximum norm C, which limits the influence of a single record. Gaussian noise is then added before the optimizer step or before update sharing. The privacy budget will be reported using epsilon and delta. Several privacy settings will be tested because stronger noise may improve privacy but reduce minority-attack recall. This privacy-utility trade-off is a central experimental question.

D. Poisoned-Update Detection

A malicious or compromised client may scale, reverse, randomize, or deliberately manipulate model parameters. PRIVATE SHIELD applies a lightweight validation stage before aggregation. For each update, the server computes its L2 distance from the current global model or from a robust reference formed by the received updates. A threshold based on the median distance and median absolute deviation can adapt to round-to-round changes. Clearly abnormal updates are rejected; borderline updates are down-weighted. The prototype will compare this method with ordinary FedAvg to measure the improvement under poisoning.

E. Reputation-Based Client Validation

Each client begins with a neutral reputation value. The score increases after timely and statistically consistent contributions and decreases after missing rounds, submitting malformed data, or repeatedly generating rejected updates. Reputation is not used as the only evidence of an attack; instead, it adjusts selection priority or aggregation weight. A recovery rule prevents one accidental failure from permanently excluding a client. This component provides a practical response to free-rider and unreliable participation concerns [11], [12].

F. SOC Dashboard and LLM-Assisted Explanation

The dashboard will display system state, device participation, message rate, current round, detected attack type, alert severity, global model metrics, privacy settings, and update-rejection events. WebSockets enable live status updates without repeatedly refreshing the page. For each important alert, structured evidence such as abnormal packet rate, suspicious port activity, update distance, and client reputation is passed to an explanation module. The generated summary should state what happened, why it was flagged, and what an administrator should inspect next. Raw packet payloads and local training records will not be sent to the language model.

V. THREAT MODEL AND SECURITY ASSUMPTIONS

The prototype assumes an honest-but-curious coordination server that follows the FL protocol but should not learn individual raw records. A subset of clients may be malicious, compromised, inactive, or selfish. Attacks considered in the first evaluation include denial-of-service traffic, port scanning, Mirai-style connection bursts, label flipping, model replacement, update scaling, random update injection, and free-rider behavior. The framework does not claim complete protection against collusion, advanced adaptive attackers, or a fully compromised server. These limitations are kept explicit so the experimental results remain realistic.

Secure transport, authentication, and access control are required in a deployment. For the academic prototype, MQTT and HTTP/WebSocket services will run in a controlled environment. Tokens or client identifiers can be used to prevent accidental unauthorized participation. Differential privacy protects individual contribution patterns, while update filtering protects model integrity; the two mechanisms solve different risks and are therefore combined.

VI. IMPLEMENTATION PLAN

The software will be organized into independent modules. Simulated IoT clients generate or replay normal and attack traffic. A logger converts messages into structured local datasets. The local training module loads client-specific data, trains the lightweight model, and applies privacy controls. The federated server manages rounds, stores model versions, validates updates, performs aggregation, and updates

reputation scores. A backend API exposes model and alert information, while a React interface provides live visualization. Experiment configuration files will define client count, data distribution, attack ratio, privacy noise, clipping norm, poisoning fraction, and rejection threshold.

Development will proceed incrementally. First, centralized and ordinary FL baselines will be implemented. Second, differential privacy will be added and tested. Third, malicious update simulation and distance-based filtering will be integrated. Fourth, reputation scoring and dashboard reporting will be connected. Finally, combined experiments will evaluate whether the full pipeline provides a useful balance of privacy, detection quality, and resilience.

VII. EVALUATION STRATEGY

A. Experimental Scenarios

Five main scenarios will be tested: centralized training, standard FedAvg, FedAvg with differential privacy, FedAvg under malicious clients, and the complete PRIVATE SHIELD pipeline. Experiments will use both balanced and non-IID client partitions. The malicious-client ratio will be increased gradually to identify the point at which global performance degrades. Multiple random seeds will be used where practical to reduce dependence on a single split.

B. Metrics

Detection quality will be measured using accuracy, precision, recall, F1-score, false-positive rate, and confusion matrices. Because attack samples may be imbalanced, macro-F1 and per-class recall will receive more attention than accuracy alone. System measurements will include training time per round, client computation time, communication volume, accepted and rejected update counts, dashboard alert latency, and the privacy budget. Robustness will be reported as the difference between clean and attacked performance.

C. Ablation and Usability Checks

Ablation experiments will remove one module at a time: differential privacy, update filtering, reputation weighting, or explanation support. This will show which component causes each improvement or cost. A small user-oriented validation can also compare raw technical alerts with structured LLM-assisted summaries. Participants can answer short questions about the likely cause and next action; completion time and correctness can indicate whether the explanations are operationally useful.

D. Validation Criteria and Baseline Comparison

The complete system will be considered useful only when its benefits can be shown against clear baselines. Standard FedAvg should provide the main distributed-learning reference, while centralized training should indicate the performance that is possible when privacy and communication constraints are ignored. PRIVATE SHIELD should preserve comparable attack recall under clean conditions, reduce the performance loss caused by malicious clients, and avoid an excessive increase in round time or bandwidth. Privacy results will be interpreted together with utility results rather than reporting epsilon alone.

Update filtering will be evaluated through true rejection of malicious updates and false rejection of honest updates. Reputation scoring will be checked for recovery after temporary client failure so that unstable connectivity is not automatically treated as malicious behavior. Dashboard validation will measure the time between an event, server-side decision, and visible alert. These criteria prevent the evaluation from relying on one headline accuracy value.

E. Reproducibility, Safety, and Ethical Use

All important experiment settings will be stored in configuration files, including random seed, dataset split, client count, local epochs, attack ratio, clipping norm, noise multiplier, and rejection threshold. Logs will preserve round-level decisions and model metrics, and code versions will be managed through Git. Attack generators will operate only on localhost or an isolated laboratory network. They will produce controlled traffic for defensive testing and will not scan or disrupt external systems. Any LLM explanation component will receive structured, minimized event information and will not be allowed to make automatic blocking decisions without visible supporting evidence.

VIII. GAP-TO-SOLUTION MAPPING

TABLE I
RESEARCH GAP TO PROPOSED SOLUTION MAPPING

Research Gap	Proposed Response and Expected Benefit
Raw-data exposure and high bandwidth	Local client training with parameter sharing only. Benefit: Reduced telemetry transfer and privacy exposure.
Gradient privacy leakage	Opacus-based differential privacy and gradient clipping. Benefit: Lower risk of reconstructing individual records.
Poisoned model updates	Euclidean-distance anomaly scoring, adaptive thresholding, rejection or down-weighting. Benefit: More robust global model under malicious clients.
Free-rider or unreliable clients	Reputation score based on contribution quality, consistency, and availability. Benefit: Fairer and more reliable participation.
Non-IID and imbalanced IoT traffic	Client-local training, controlled partitions, macro-F1 and per-class evaluation. Benefit: More realistic measurement of distributed performance.
Limited SOC explainability	React dashboard, WebSocket alerts, and structured LLM forensic summaries. Benefit: Faster understanding and response.
Static security evaluation	Continuous attack simulation and round-level adversarial stress testing. Benefit: Detection of runtime model degradation.

IX. EXPECTED CONTRIBUTIONS AND LIMITATIONS

The first expected contribution is a complete educational prototype that demonstrates how privacy and robustness can coexist in a federated IoT IDS. The second is an experimental comparison of privacy noise, poisoning defense, and non-IID

behavior within the same codebase. The third is an operational bridge between distributed model training and an SOC interface. The fourth is a documented attack-simulation and evaluation workflow that can be reproduced by future student teams.

The project has several limitations. Simulated devices cannot reproduce every timing, power, and hardware constraint of real IoT nodes. Euclidean distance is a lightweight defense but may not detect carefully crafted adaptive attacks. Differential privacy may reduce recall for rare attacks. Reputation values can be affected by unstable networks and must not be interpreted as proof of malicious intent. LLM summaries may be incomplete or incorrect, so the dashboard must show the underlying evidence and treat generated text as assistance rather than an automatic final decision.

X. CONCLUSION

PRIVATE SHIELD is proposed as a unified privacy-preserving and adversarial-aware framework for IoT threat detection. It replaces centralized raw-data collection with federated local training, protects updates through clipping and differential privacy, validates model contributions before aggregation, reduces the influence of unreliable clients through reputation scoring, and presents readable security information through a live dashboard. The design directly addresses gaps identified across prior work instead of treating privacy, poisoning, participation, and explainability as unrelated problems. A structured evaluation against centralized and standard FL baselines will determine whether the full framework improves practical resilience while preserving acceptable detection performance and system overhead.

REFERENCES

[1] I. A. Soomro, H. U. Rehman, S. J. Hussain, A. Iqbal, W. Khalid, and H. Yu, “SecureDyn-FL: A Robust Privacy-Preserving Federated Learning Framework for Intrusion Detection in IoT Networks,” arXiv:2601.06466, 2026.

[2] S. Izadi and M. Ahmadi, “Adaptive Meta-Aggregation Federated Learning for Intrusion Detection in Heterogeneous Internet of Things,” arXiv:2602.12541, 2026.

[3] S. Izadi and M. Ahmadi, “Lightweight Cluster-Based Federated Learning for Intrusion Detection in Heterogeneous IoT Networks,” arXiv:2602.12543, 2026.

[4] G. Singh, K. Sood, P. Rajalakshmi, and Y. Xiang, “Sentinel: Dynamic Knowledge Distillation for Personalized Federated Intrusion Detection in Heterogeneous IoT Networks,” arXiv:2510.23019, 2025.

[5] J. Shen, W. Yang, Z. Chu, J. Fan, D. Niyato, and K.-Y. Lam, “Effective Intrusion Detection in Heterogeneous Internet-of-Things Networks via Ensemble Knowledge Distillation-based Federated Learning,” arXiv:2401.11968, 2024.

[6] S. Sun, P. Sharma, K. Nwodo, A. Stavrou, and H. Wang, “FedMADE: Robust Federated Learning for Intrusion Detection in IoT Networks Using a Dynamic Aggregation Method,” arXiv:2408.07152, 2024.

[7] N. Rattanavipanon and I. De Oliveira Nunes, “Poisoning Prevention in Federated Learning and Differential Privacy via Stateful Proofs of Execution,” arXiv:2404.06721, 2024.

[8] A. ElZemity and B. Arief, “Privacy Threats and Countermeasures in Federated Learning for Internet of Things: A Systematic Review,” arXiv:2407.18096, 2024.

[9] E. Mármol Campos, A. González Vidal, J. L. Hernández Ramos, and A. Skarmeta, “FedRDF: A Robust and Dynamic Aggregation Function against Poisoning Attacks in Federated Learning,” arXiv:2402.10082, 2024.

[10] P. Mai, R. Yan, and Y. Pang, “RFLPA: A Robust Federated Learning Framework against Poisoning Attacks with Secure Aggregation,” arXiv:2405.15182, 2024.

[11] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, “PASS: A Parameter Audit-based Secure and Fair Federated Learning Scheme against Free-Rider Attack,” arXiv:2207.07292, 2022.

[12] J. Chen, M. Li, T. Liu, H. Zheng, Y. Cheng, and C. Lin, “Rethinking the Defense Against Free-rider Attack From the Perspective of Model Weight Evolving Frequency,” arXiv:2206.05406, 2022.